National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 7: Transfer Learning Report

Feature Extraction vs. Fine-Tuning on CIFAR-10

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 11, 2026

1 Introduction

While utilizing pre-trained networks is a highly efficient deep learning strategy, there are two distinct paradigms for adapting these models to new datasets: **Feature Extraction** and **Fine-Tuning**.

- **Feature Extraction:** The structural weights of the pre-trained base model are completely "frozen." Only the newly initialized final classification layer is trained. The model acts purely as a static feature extractor.
- **Fine-Tuning:** After initial feature extraction, the top layers of the pre-trained base model are "unfrozen." These layers are then jointly trained with the classifier using an extremely low learning rate, allowing the model to adapt its learned high-level features specifically to the new target dataset.

Table 1: Core Differences Between Transfer Learning Paradigms

Attribute	**Feature Extraction**	**Fine-Tuning**
Base Weights	Frozen (Static)	Unfrozen (Updated)
Learning Rate	Standard (10^{-3})	Micro (10^{-5})
Training Speed	Fast	Slower
Risk of Overfitting	Low	High (if not regularized)

This tutorial evaluates both methodologies using the CIFAR-10 dataset, comparing a Feature Extraction approach (ResNet50) against a Fine-Tuning approach (VGG16).

2 Methodology

Both architectures were implemented using PyTorch and trained on a T4 GPU. The CIFAR-10 dataset was resized to 224×224 and normalized to align with the ImageNet distributions.

2.1 Task A: ResNet50 (Feature Extraction)

A pre-trained ResNet50 model was loaded, and all base parameters were frozen by setting `requires_grad = False`. The fully connected (`fc`) layer was replaced to output 10 classes. The network was trained for **3 epochs** using the Adam optimizer with a standard learning rate of 10^{-3}.

2.2 Task B: VGG16 (Fine-Tuning)

The VGG16 model underwent a two-stage training process:

1. **Stage 1 (Feature Extraction):** All base layers were frozen. The classifier was trained for **2 epochs** at a learning rate of 10^{-3}.

2. **Stage 2 (Fine-Tuning):** The final 5 convolutional layers of the feature block were unfrozen. Crucially, the learning rate was dropped to a micro-level (10^{-5}) to gently update the weights without causing catastrophic forgetting. The model was trained for an additional **3 epochs**.

```
# 1. Unfreeze the final 5 layers of the VGG16 feature block
for param in vgg_model.features[-5:].parameters():
param.requires_grad = True

# 2. Lower the learning rate drastically to prevent weight destruction
optimizer = optim.Adam(
filter(lambda p: p.requires_grad, vgg_model.parameters()),
lr=1e-5
)

```

Listing 1: PyTorch implementation for unfreezing VGG16 layers and applying a micro-learning rate.

3 Results and Visual Analysis

As illustrated in Figure 1 and Table 2, the fine-tuning methodology yielded significantly higher accuracy.

Table 2: Final Validation Accuracy Comparison

Model Structure	Training Approach	Final Validation Accuracy
ResNet50	Feature Extraction	81.48%
VGG16	Fine-Tuned	**87.72%**

3.1 Performance Breakdown

- **ResNet50 (Feature Extraction):** The model plateaued rapidly, reaching its peak capability as a static feature extractor at **81.48%**.
- **VGG16 (Fine-Tuned):** While the model reached ∼82% during the static feature extraction phase, unfreezing the layers and applying the 10^{-5} learning rate caused an immediate surge in performance, driving the final validation accuracy up to **87.72%**.

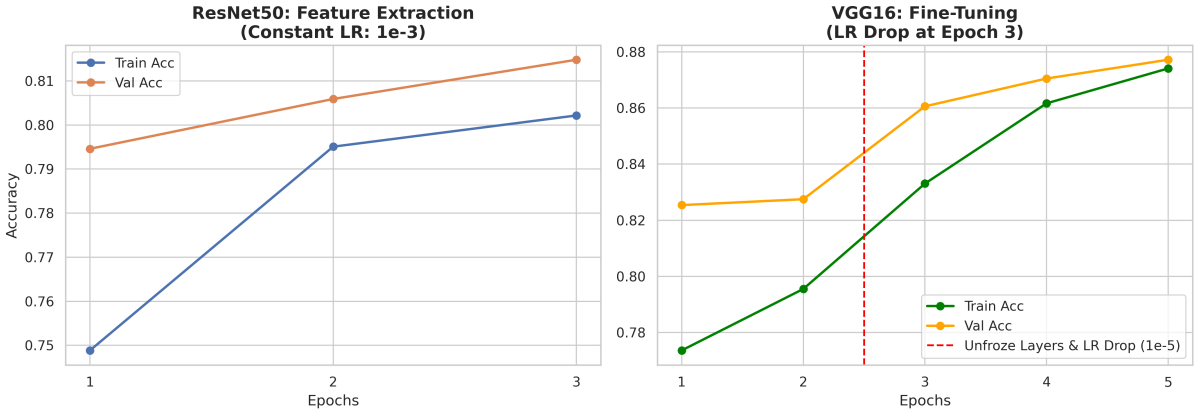

Figure 1: Training and Validation curves demonstrating the impact of Fine-Tuning. The dashed line clearly shows the immediate accuracy gain upon unfreezing layers.

4 Strategies to Improve Fine-Tuning

While fine-tuning is powerful, unfreezing layers increases the risk of overfitting, especially on smaller datasets. The following strategies can be implemented to further improve fine-tuning results:

- **Modulating the Learning Rate:** Pre-trained weights are fragile. Using an excessively high learning rate will destroy the learned features. Utilizing a learning rate scheduler (e.g., `ReduceLROnPlateau`) ensures the model makes increasingly smaller, refined updates.
- **Adjusting Unfrozen Layers:** The optimal number of layers to unfreeze depends on dataset similarity. If the new dataset is vastly different from ImageNet, unfreezing more blocks is beneficial. If the dataset is small, unfreezing too many layers will result in immediate overfitting.
- **Early Stopping:** Because fine-tuned models can memorize small datasets quickly, monitoring the validation loss and halting training the moment it diverges from the training loss prevents over-parameterization.
- **Aggressive Regularization (Dropout):** Adding Dropout layers (e.g., $p = 0.5$) before the fully connected head forces the network to learn redundant representations, further preventing the fine-tuned layers from overfitting to noise.

5 Conclusion

This tutorial successfully demonstrated the empirical advantage of fine-tuning over static feature extraction. While freezing layers offers computational speed, unfreezing the top convolutional blocks and applying a micro-learning rate (10^{-5}) allows the network to adapt its spatial filters to the specific domain of the new data. Consequently, the fine-tuned VGG16 model outperformed the feature-extracted ResNet50 model by over **6%**, proving it to be the superior method for maximizing transfer learning accuracy.